Web Application Basics

Basics of web applications: HTTP, URLs, request methods, response codes, and headers.

Web Application Basics provides a comprehensive foundation for understanding how modern web services operate. The module begins by dissecting the architecture of web applications, distinguishing between **Front End** technologies (HTML, CSS, JavaScript) and **Back End** infrastructure (Databases, Web Servers, and WAFs). It provides a granular breakdown of the **Uniform Resource Locator (URL)** and the structure of **HTTP Messages**, including requests and responses. Key concepts such as **HTTP Methods** (GET, POST, etc.), **Status Codes**, and **Headers** are explained with a focus on their functional and security implications. Finally, the content introduces critical **Security Headers** like CSP and HSTS, which serve as modern defenses against common web-based vulnerabilities.

This **documentation** and **learning approach** is strongly inspired by the structured, hands-on content provided by TryHackMe, which significantly contributed to practical understanding of Public Key Cryptography Basics concepts.

WEIRDO

DOCUMENTED BY

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

1. Web Application Architecture

A web application is a software program that runs on a web server and is accessed through a web browser. It is divided into two primary environments:

Front End (Client-Side)

The "surface" that the user interacts with.

- **HTML (Hypertext Markup Language):** The structural skeleton of the page.
- **CSS (Cascading Style Sheets):** Defines the visual style, layout, and colors.
- **JavaScript (JS):** The "brain" that enables interactivity and logic within the browser.

Back End (Server-Side)

The "engine" that handles data processing and storage.

- **Web Server:** Processes incoming requests and delivers content (e.g., Nginx, Apache).
- **Database:** A structured system to store and retrieve data (e.g., SQL, NoSQL).
- **WAF (Web Application Firewall):** A security layer that filters malicious traffic before it reaches the server.

2. Anatomy of a URL (Uniform Resource Locator)

A URL is a specific address used to access resources on the Internet. It consists of several distinct components:

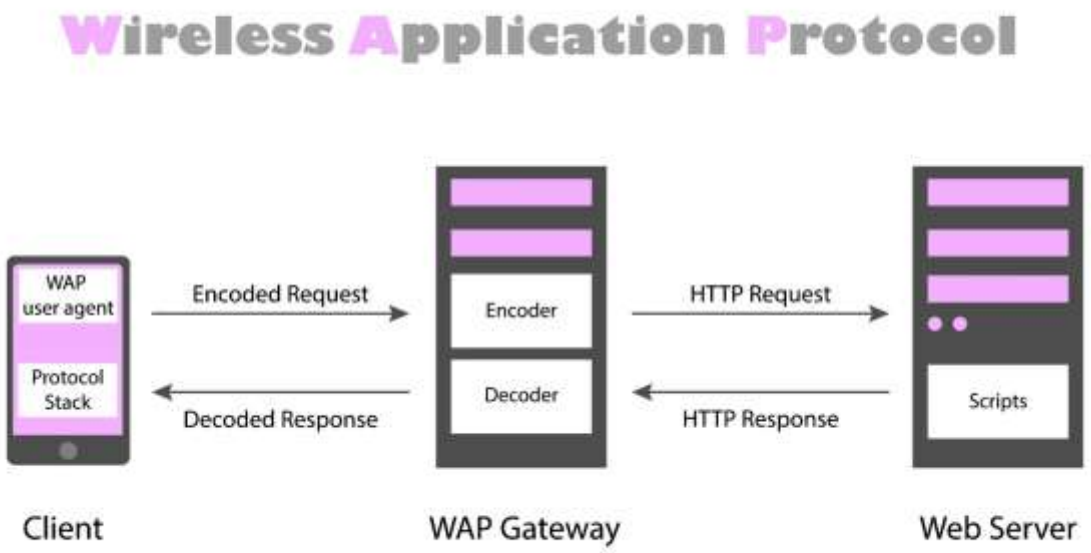
Component	Example	Description
Scheme	**https://**	The protocol used (HTTP or HTTPS).
User	**admin@**	Optional authentication credentials.
Host/Domain	**tryhackme.com**	The unique name of the target website.
Port	**:443**	The communication "doorway" (Default 80 for HTTP, 443 for HTTPS).
Path	**/api/users**	The specific location of a file or resource on the server.
Query String	**?id=123**	Parameters used to pass data (e.g., search terms).
Fragment	**#section1**	Points to a specific location within a webpage.

WEIRDO

3. HTTP Messages: Requests and Responses

Communication between a client and a server is conducted via **HTTP Messages**. These follow a strict four-part format:

1. **Start Line:** Defines the message type (Method for requests, Status for responses).
2. **Headers:** Key-value pairs providing metadata and instructions.
3. **Empty Line:** A mandatory divider between headers and the body.
4. **Body:** The actual data (HTML, JSON, or form input).



5. HTTP Request Methods and Versions

Common Request Methods

- **GET:** Fetches data; should never be used for sensitive information.
- **POST:** Submits data to create or update a resource.
- **PUT:** Replaces an existing resource entirely.
- **DELETE:** Removes a resource from the server.
- **OPTIONS:** Returns the allowed communication methods for a resource.
- **HEAD:** Same as GET but retrieves only the headers (no body).

Protocol Versions

- **HTTP/1.1:** Introduced persistent connections; still widely used.
- **HTTP/2:** Focuses on speed via multiplexing and header compression.
- **HTTP/3:** Uses the **QUIC** protocol for faster, more secure connections over UDP.

5. Request Headers and Body Formats

Common Request Headers

- **Host:** The domain name of the server.
- **User-Agent:** Identifies the browser/OS of the client.
- **Referer:** The URL of the previous page that linked to the current request.
- **Cookie:** Holds session data previously set by the server.
- **Content-Type:** Specifies the format of the data in the request body.

Body Data Types

- **URL Encoded:** key1=value1&key2=value2 (Standard for simple forms).
- **Form Data:** Used for binary files and images; uses **boundary strings**.
- **JSON:** { "key": "value" } (Standard for modern APIs).
- **XML:** Structured tags <key>value</key>.

6. HTTP Response: Status Codes and Headers

Status Code Categories

The **Status Line** indicates the result of the request:

Range	Category	Description
100-199	Informational	Request received, continuing process.
200-299	**Success**	The action was successfully received and accepted.
300-399	Redirection	Further action is needed to complete the request.
400-499	**Client Error**	The request contains bad syntax or cannot be fulfilled.
500-599	**Server Error**	The server failed to fulfill an apparently valid request.

Security-Specific Response Headers

- **Set-Cookie:**
  - **HttpOnly:** Prevents JavaScript from accessing the cookie (mitigates XSS).
  - **Secure:** Ensures the cookie is only sent over encrypted HTTPS.
- **Server:** Reveals the web server software (should be obscured for security).

WEIRDO

7. Security Headers

Modern browsers use specific headers to enforce security policies:

- **Content-Security-Policy (CSP):** Restricts the domains from which scripts and resources can be loaded.
- **Strict-Transport-Security (HSTS):** Forces the browser to use HTTPS exclusively.
- **X-Content-Type-Options:** Set to **nosniff** to prevent browsers from guessing MIME types.
- **Referrer-Policy:** Controls how much information is shared when clicking a link to another site.

Conclusion and Cheatsheet

Conclusion

Understanding Web Application Basics is the first step toward web security and development. By mastering the relationship between the **Front End** and **Back End**, the structure of **URLs**, and the mechanics of **HTTP messaging**, one can identify how data moves across the internet. Recognizing **Status Codes** and implementing **Security Headers** are vital skills for maintaining robust and secure web applications in a modern threat landscape.

Fast Revision Cheatsheet

- **Port 80:** HTTP (Unencrypted).
- **Port 443:** HTTPS (Encrypted).
- **Method GET:** Read-only data.
- **Method POST:** Data submission.
- **200 OK:** Success.
- **301 Moved Permanently:** Permanent redirect.
- **404 Not Found:** Resource missing.
- **500 Internal Server Error:** Server-side crash.
- **HttpOnly Flag:** Blocks JS access to cookies.
- **Secure Flag:** Forces HTTPS for cookies.
- **nosniff:** Prevents MIME-type sniffing.